CJAVA

siempre para apoyarte

Visión

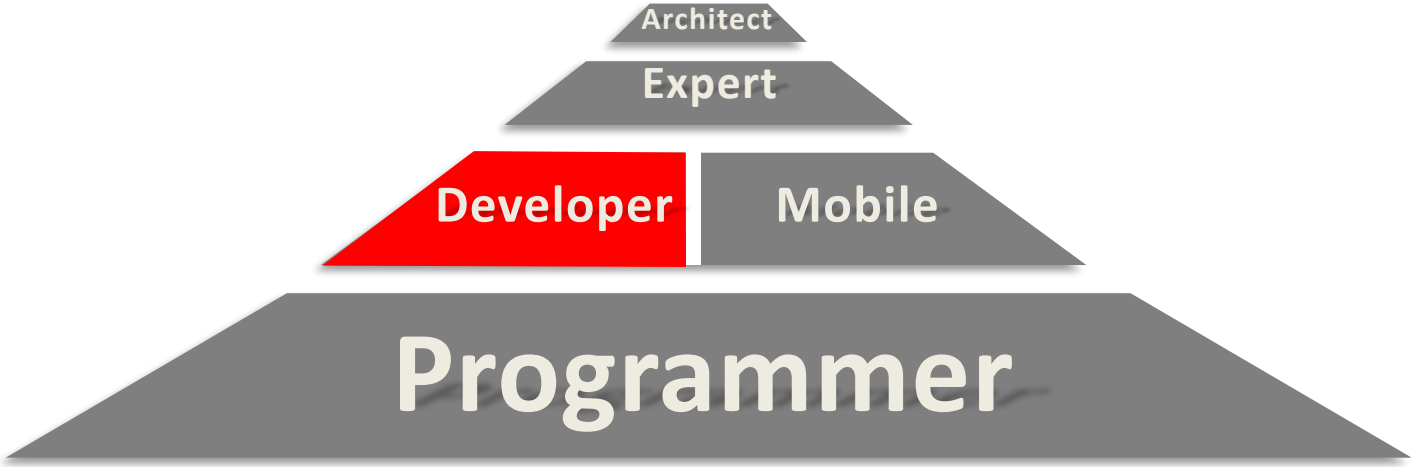
Poder aportar al desarrollo del País usando tecnología Java.

Edwin Maraví
emaravi@cjavaperu.com

CJAVA

CJAVA
siempre para apoyarte

CJava



FrameWork

Edwin Maraví

emaravi@cjavaperu.com

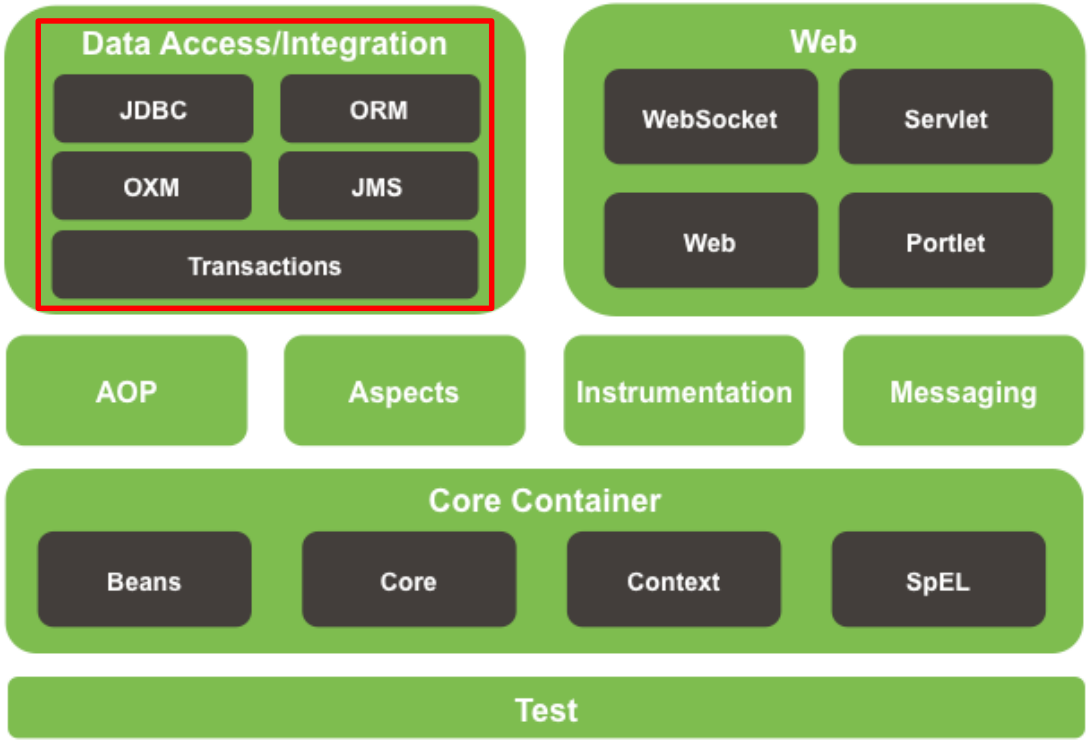
CJAVA
siempre para apoyarte

Objetivos

- Aprender conceptos básicos de Spring JDBC
- Aprender conceptos básicos de Spring Framework

Administración de Transacciones

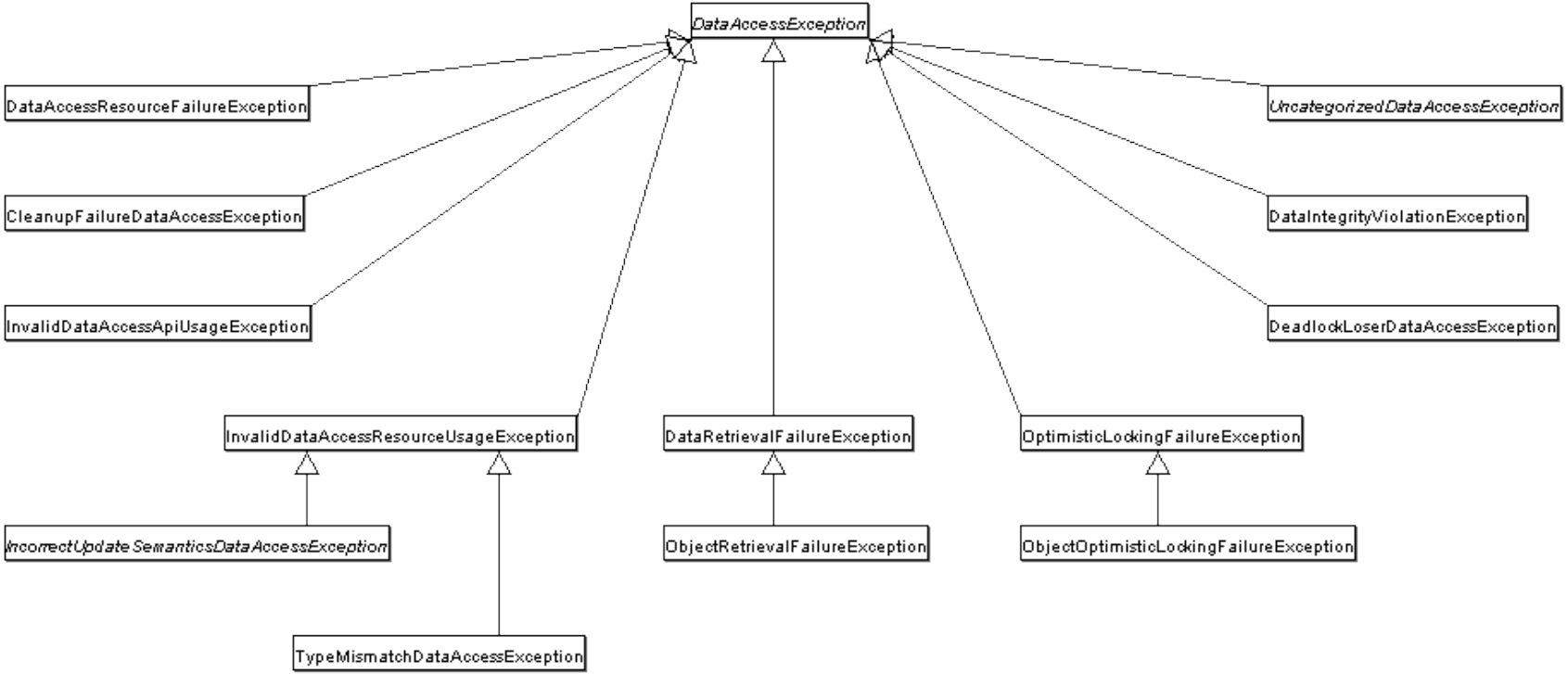
Data Access / Integration



DAO Support

El soporte del objeto de acceso a datos (DAO - Data Access Object) en Spring se centra en la facilidad de trabajar con tecnologías de acceso a datos como JDBC, Hibernate, JPA o JDO de una manera consistente. Esto permite cambiar entre las tecnologías de persistencia citadas con bastante facilidad y también le permite a uno el código sin tener que preocuparse por la captura de excepciones que son específicas de cada tecnología.

Jerarquía de Excepciones



Anotaciones para configurar DAO

La mejor manera de garantizar que sus objetos de acceso a datos (DAO) o repositorios traducen correctamente la excepción, es el uso de la anotación **@Repository**. Esta anotación también permite el apoyo de escaneo componente para encontrar y configurar sus DAOs sin tener configurarlas en el XML.

```
@Repository
public class SomeMovieFinder implements MovieFinder {
    // ...
}
```

Anotaciones para configurar DAO

Ejemplo para JPA:

```
@Repository
public class JpaMovieFinder implements MovieFinder {

    @PersistenceContext
    private EntityManager entityManager;

    // ...

}
```

Ejemplo para Hibernate:

```
@Repository
public class HibernateMovieFinder implements MovieFinder {

    private SessionFactory sessionFactory;

    @Autowired
    public void setSessionFactory(SessionFactory sessionFactory) {
        this.sessionFactory = sessionFactory;
    }

    // ...

}
```

Anotaciones para configurar DAO

Ejemplo para Spring JDBC:

```
@Repository
public class JdbcMovieFinder implements MovieFinder {

    private JdbcTemplate jdbcTemplate;

    @Autowired
    public void init(DataSource dataSource) {
        this.jdbcTemplate = new JdbcTemplate(dataSource);
    }

    // ...

}
```

Clases de Soporte DAO

- **JdbcDaoSupport**

  Requisito: DataSource. (+) JdbcTemplate

- **HibernateDaoSupport** (Hibernate v3 y v4)

  Requisito: SessionFactory. (+) HibernateTemplate

- **CciDaoSupport (**Common Client Interface for EIS**)**

  Requisito: ConnectionFactory (+) CciTemplate

CJAVA
siempre para apoyarte

Spring JDBC

Acción	Spring	Usted
Definir los parámetros de conexión.		X
Abrir conexión.	X	
Especificar la sentencia SQL		X
Declarar parámetros y proporcionar valores.		X
Preparar y ejecutar la sentencia.	X	
Configurar el bucle para recorrer la iteración de los resultados (si los hay)	X	
Hacer el trabajo para cada iteración.		X
Procesar cualquier excepción.	X	
Manejar las transacciones.	X	
Cerrar la conexión, estado de cuenta y de resultados.	X	

CJAVA
siempre para apoyarte

Clases de Soporte JDBC

- **JdbcTemplate** :

  Implementación básica de Spring JDBC.

- **NamedParameterJdbcTemplate:**

  Permite usar nombres para los parámetros en vez del comodín (?)

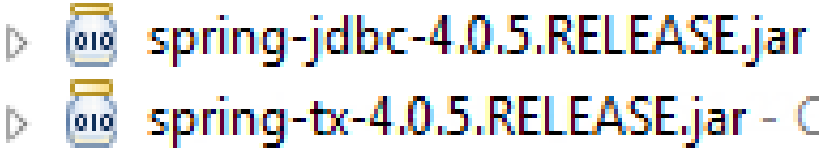
- **SimpleJdbcInsert y SimpleJdbcCall**

  Recibe un Map de parámetros y el nombre de una tabla o StoredProcedure y realiza una comparación entre los nombres de las columnas.

- **SqlUpdate, StoredProcedure, etc**

Librerias JDBC

- **Librerías necesarias:** estas son adicionales a las librerías ya usadas en las sesiones anteriores a excepción de las librerías **web** y **webmvc** de Spring y las que se utiliza para web.



Spring JDBC

Ejecutando una consulta con **JdbcTemplate**:

```
@Repository
public class JdbcTemplateDao {

    private JdbcTemplate jdbcTemplate;

    @Autowired
    public void setDataSource(DataSource dataSource) {
        this.jdbcTemplate = new JdbcTemplate(dataSource);
    }

    public Persona getPersona(Integer id){
        try{
            String sql = "select id, nombres, apellidos, edad from persona where id=?";
            return jdbcTemplate.queryForObject(sql,
                                               ParameterizedBeanPropertyRowMapper.newInstance(Persona.class),
                                               id);
        }catch(EmptyResultDataAccessException e){}
        return null;
    }
```

Spring JDBC

Ejecutando una consulta con **NamedParameterJdbcTemplate**:

```
@Repository
public class NamedParameterJdbcTemplateDao {

    private NamedParameterJdbcTemplate namedParameterJdbcTemplate;

    @Autowired
    public void setDataSource(DataSource dataSource) {
        this.namedParameterJdbcTemplate = new NamedParameterJdbcTemplate(dataSource);
    }

    public Persona getPersona(Integer id){
        Map<String, Object> parametros = new HashMap<String, Object>();
        parametros.put("id", id);
        try{
            String sql = "select id, nombres, apellidos, edad from persona where id=:id";
            return namedParameterJdbcTemplate.queryForObject(sql,
                                                             parametros,
                                                             ParameterizedBeanPropertyRowMapper.newInstance(Persona.class));
        }catch(EmptyResultDataAccessException e){}
        return null;
    }
```

Spring JDBC

Ejecutando una consulta con **JdbcDaoSupport**:

```
@Repository
public class JdbcDaoSupportDao extends JdbcDaoSupport {

    @Autowired
    public void setDS(DataSource dataSource){
        super.setDataSource(dataSource);
    }

    public Persona getPersona(Integer id){
        try{
            String sql = "select id, nombres, apellidos, edad from persona where id=?";
            return getJdbcTemplate().queryForObject(sql,
                                                    ParameterizedBeanPropertyRowMapper.newInstance(Persona.class),
                                                    id);
        }catch(EmptyResultDataAccessException e){}
        return null;
    }

}
```

Spring JDBC

Ejecutando un Cursor con **SimpleJdbcCall**:

```
@Repository
public class SimpleJdbcCallDao {

    private SimpleJdbcCall simpleJdbcCall;

    @Autowired
    public void setDataSource(DataSource dataSource) {
        this.simpleJdbcCall = new SimpleJdbcCall(dataSource);
    }

    public List<Persona> buscar(Integer id){
        try{
            SqlParameterSource parametrosIN = new MapSqlParameterSource()
            .addValue("P_ID", id, Types.INTEGER);

            Map<String, Object> parametrosOUT = simpleJdbcCall
            .withProcedureName("SP_GET_PERSONAS")
            .declareParameters(
                    new SqlParameter("P_ID", Types.INTEGER),
                    new SqlOutParameter("P_TOTAL", Types.INTEGER))
            .returningResultSet("#result-set-1", ParameterizedBeanPropertyRowMapper.newInstance(Persona.class))
            .execute(parametrosIN);
            System.out.println("OUTPUT: " + parametrosOUT);
            System.out.println("tota: " + parametrosOUT.get("P_TOTAL"));
            return (List<Persona>) parametrosOUT.get("#result-set-1");
        }catch(EmptyResultDataAccessException e){}
        return null;
    }
```

Spring JDBC

Uso de **ResultSetExtractor** y **RowMapper** para mapear el resultado de una consulta usando un query o por cursor:

```
public class PersonaExtractor implements ResultSetExtractor<Persona> {

    @Override
    public Persona extractData(ResultSet res) throws SQLException, DataAccessException {
        //Para el caso de un solo registro cuando se llama directamente al extractor
        if(res.isBeforeFirst()) res.next();
        return new Persona(res.getInt("id"),
                           res.getString("nombres"),
                           res.getString("apellidos"),
                           res.getInt("edad"));
    }
}
```

```
public class PersonaRowMapper implements RowMapper<Persona> {

    @Override
    public Persona mapRow(ResultSet res, int line) throws SQLException {
        return new PersonaExtractor().extractData(res);
    }
}
```

Spring Transaction Management

Teoría de las Transacciones:

En la tradición del desarrollo de software se creo un acrónimo para describir las transacciones. ACID lo cual significa:

- **Atomic (Atómica):** Las transacciones se componen de una o mas actividades unidas como una única unidad de trabajo. La atomicidad asegura que todas o ningunas de las operaciones de una transacción tiene lugar.
- **Consistent (Consistente):** Una vez que termina una transacción (con o sin éxito) los datos no deben estar corruptos con relación a la realidad.
- **Isolated (Aislada):** Las transacciones deben estar aisladas una de otras, evitando que ocurran lecturas concurrentes e introduccion de los mismos datos (el aislamiento típicamente implica el bloqueo de filas y/o tablas de una base de datos).
- **Durable (Duradera):** Una vez la transacción se ha completado, los resultados deben hacerse permanentes de forma que sobrevivan. Esto implica almacenar los resultados en una base de datos u otro tipo de almacenamiento persistente.

CJAVA
siempre para apoyarte

Gestores de Transacción

Spring no gestiona las transacciones directamente. En lugar de eso, contiene una selección de gestores de transacción que delegan la responsabilidad de la gestión de transacción a una implementación de transacción de una plataforma específica. Algunos gestores de Spring mas conocidos:

Gestor de Transacción	Usado
jdbc.datasource.DataSourceTransactionManager	Se trabaja con el soporte de abstracción JDBC de Spring. También utilizado para IBATIS.
orm.hibernate3.HibernateTransactionManager	Se usa Hibernate3 para persistencia.
orm.toplink.TopLinkTransactionManager	Se utiliza TopLink de Oracle para persistencia.

Configurando la Transacción

Existen 3 formas de configurar las transacciones en Spring:

- Definición Programática
- Definición Declarativa – Anotaciones
- Definición Declarativa – XML (AOP)

Configurando la Transacción

Definición Programática:

```java
@Service
public class PersonaServiceProgramaticaImpl{

    @Autowired
    private TransactionTemplate transactionTemplate;

    @Autowired
    private PersonaDao personaDao;

    public int insertar(Persona persona) {
        final Persona personaFinal = persona;
        int resultado = transactionTemplate.execute(new TransactionCallback<Integer>() {
            @Override
            public Integer doInTransaction(TransactionStatus ts) {
                try{
                    return personaDao.insertar(personaFinal);
                }catch(Exception e){
                    System.out.println("Se produjo un error: " + e.getMessage());
                    ts.setRollbackOnly();
                }
                return 0;
            }

        });
        return resultado;
    }

}
```

Configurando la Transacción

Definición Programática:

```xml
<?xml version="1.0"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:tx="http://www.springframework.org/schema/tx"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
                        http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
                        http://www.springframework.org/schema/context
                        http://www.springframework.org/schema/context/spring-context-4.0.xsd
                        http://www.springframework.org/schema/tx
                        http://www.springframework.org/schema/tx/spring-tx-4.0.xsd">

    <context:component-scan base-package="com.cjavaperu" />

    <context:property-placeholder location="jdbc.properties" />

    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource"
        destroy-method="close">
        <property name="driverClassName" value="${jdbc.driverClassName}" />
        <property name="url" value="${jdbc.url}" />
        <property name="username" value="${jdbc.username}" />
        <property name="password" value="${jdbc.password}" />
    </bean>

    <bean id="txManager" class="org.springframework.jdbc.datasource.DataSourceTransactionManager" >
        <property name="dataSource" ref="dataSource" />
    </bean>

    <bean id="transactionTemplate" class="org.springframework.transaction.support.TransactionTemplate" >
        <property name="transactionManager" ref="txManager" />
    </bean>

</beans>
```

Necesitas Inyectar TransactionTemplate a tu servicio

Configurando la Transacción

Definición Declarativa con Anotaciones:

```
@Service
public class PersonaServiceImpl implements PersonaService {

    @Autowired
    PersonaDao personaDao;

    @Override
    @Transactional(propagation = Propagation.REQUIRED, readOnly = false)
    public int insertar(Persona persona) {
        int resultado = personaDao.insertar(persona);
        return resultado;
    }
```

*Tanto en la programática como en las declarativas necesitas de un **TransactionManager**, pero en las declarativas no se necesita inyectar **TransactionTemplate.** Además cuando se utiliza la anotación @Transactional se necesita agregar la etiqueta **annotation-driven** indicando el **TransactionManager**.

Configurando la Transacción

Definición Declarativa con Anotaciones:

```xml
<?xml version="1.0"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:tx="http://www.springframework.org/schema/tx"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
                        http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
                        http://www.springframework.org/schema/context
                        http://www.springframework.org/schema/context/spring-context-4.0.xsd
                        http://www.springframework.org/schema/tx
                        http://www.springframework.org/schema/tx/spring-tx-4.0.xsd">

    <context:component-scan base-package="com.cjavaperu" />

    <context:property-placeholder location="jdbc.properties" />

    <tx:annotation-driven transaction-manager="txManager" />

    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource"
        destroy-method="close">
        <property name="driverClassName" value="${jdbc.driverClassName}" />
        <property name="url" value="${jdbc.url}" />
        <property name="username" value="${jdbc.username}" />
        <property name="password" value="${jdbc.password}" />
    </bean>

    <bean id="txManager" class="org.springframework.jdbc.datasource.DataSourceTransactionManager" >
        <property name="dataSource" ref="dataSource" />
    </bean>

</beans>
```

Configurando la Transacción

Definición Declarativa con AOP:

No se necesita agregar la anotación **@Transactional**

```java
@Service
public class PersonaServiceAOPImpl{

    @Autowired
    private PersonaDao personaDao;

    public int guardarPersona(Persona persona) {
        int resultado = personaDao.insertar(persona);
        return resultado;
    }

}
```

Librería necesaria para trabajar con aspectos:

aspectjweaver-1.8.1.jar

```xml
<?xml version="1.0"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:tx="http://www.springframework.org/schema/tx"
    xmlns:aop="http://www.springframework.org/schema/aop"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
                        http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
                        http://www.springframework.org/schema/context
                        http://www.springframework.org/schema/context/spring-context-4.0.xsd
                        http://www.springframework.org/schema/tx
                        http://www.springframework.org/schema/tx/spring-tx-4.0.xsd
                        http://www.springframework.org/schema/aop
                        http://www.springframework.org/schema/aop/spring-aop-4.0.xsd">

    <context:component-scan base-package="com.cjavaperu" />

    <context:property-placeholder location="jdbc.properties" />

    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource"
        destroy-method="close">
        <property name="driverClassName" value="${jdbc.driverClassName}" />
        <property name="url" value="${jdbc.url}" />
        <property name="username" value="${jdbc.username}" />
        <property name="password" value="${jdbc.password}" />
    </bean>

    <bean id="txManager"
        class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
        <property name="dataSource" ref="dataSource" />
    </bean>

    <tx:advice id="txAdvice" transaction-manager="txManager">
        <tx:attributes>
            <tx:method name="buscar*" read-only="true" propagation="SUPPORTS" />
            <tx:method name="guardar*" read-only="false" propagation="REQUIRED"
                isolation="DEFAULT" rollback-for="Throwable" />
        </tx:attributes>

    </tx:advice>

    <aop:config>
        <aop:pointcut id="serviceOperation"
            expression="execution(* com.cjavaperu.service.*Service*.*(..))" />
        <aop:advisor pointcut-ref="serviceOperation" advice-ref="txAdvice" />
    </aop:config>

</beans>
```

Atributos de la Transacción

En Spring, las transacciones declarativas se definen con atributos.

Un atributo es una descripción de como deben aplicarse al método las políticas de transacción. Hay 5 atributos:

1. Propagación (propagation) -> Asociar la transacción al método que se ejecuta.
2. Aislamiento (isolation)
3. Solo lectura (readOnly)
4. Intervalo (timeOut)
5. Reglas de eliminación (rollBackFor, rollBackForClassName, noRollBackFor, noRollBackForClassName)

Propagación de la Transacción

Propagación	Acción
MANDATORY	Indica que el método debe ejecutarse dentro de una transacción. Si no hay transacción en proceso, se lanzará una excepción.
NESTED	Indica que el método debe ejecutarse dentro de una transacción anidada si hay una transacción en progreso. La transacción anidada puede consignarse y eliminarse independientemente de la transacción que la contiene. Si no hay transacción que la contenga se comporta como REQUIRED.
NEVER	Indica que el método no debe ejecutarse dentro de un contexto transaccional. Si hay una transacción existente en progreso, se lanzará una excepción.
NOT_SUPPORTED	Indica que el método no debe ejecutarse dentro de una transacción. Si hay una transacción en progreso, se suspenderá durante la duración del método.
REQUIRED	Indica que el método debe ejecutarse dentro de una transacción. Si hay una transacción en progreso, el método se ejecutará dentro de dicha transacción. Sino, se iniciará una nueva transacción. (POR DEFECTO)
REQUIRES_NEW	Indica que el método debe ejecutarse dentro de su propia transacción. Se inicia una nueva transacción y si hay otra en proceso se suspenderá durante la duración del método.
SUPPORTS	Indica que el método se ejecutara dentro de la transacción existente (como el REQUIRED). Si no existe una transacción no se creara una. (como el NOT_SUPPORTED).

Aislamiento de la Transacción

Propagación	Acción
DEFAULT	Lo determina la fuente de datos.
READ_UNCOMMITED	Permite leer cambios que no han sido commitiados en otras transacciones. Puede resultar en lecturas sucias, lecturas no repetibles y lecturas fantasmas.
READ_COMMITED	Permite leer cambios concurrentes que han sido commitiados. Se evitan las lecturas sucias, pero pueden ocurrir lecturas no repetibles(una misma consulta puede retornar valores diferentes dentro de la misma transacción) y lecturas fantasmas (si se hace rollback en otras transacciones).
REPEATABLE_READ	Se evitan las lecturas sucias y no repetibles pero pueden ocurrir lecturas fantasmas.
SERIALIZABLE	Se evitan las lecturas sucias , no repetibles y las lecturas fantasmas.

Links de Referencia

Web Spring

- http://spring.io

Spring Framework

- http://projects.spring.io/spring-framework

Spring Framework Data Access

- http://docs.spring.io/spring/docs/current/spring-framework-reference/html/spring-data-tier.html

CJAVA

siempre para apoyarte

Gracias